The Journal of Open Source Software

LAS-NoCode Rule Editor: A visual low-code framework for reproducible Complex Event Processing rule design in IoT research

Rosiberto Gonçalves [1¶]

1 Estácio University Center Recife, Brazil ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

Summary

Complex Event Processing (CEP) supports real-time detection of patterns and anomalies over continuous event streams and is widely adopted in Internet of Things (IoT), smart environments, and distributed systems. Despite mature execution engines, rule definition typically depends on textual domain-specific languages such as Event Processing Language (EPL), which increases cognitive load and introduces variability in rule specification.

LAS-NoCode Rule Editor (LAS) is an open-source, containerized, low-code web application that enables structured visual construction of CEP rules. LAS represents rules as directed block graphs and automatically generates syntactically valid EPL compatible with Perseo CEP within the FIWARE ecosystem. The system is designed to support reproducible experimentation, structured rule modeling, and educational adoption in event-driven systems research.

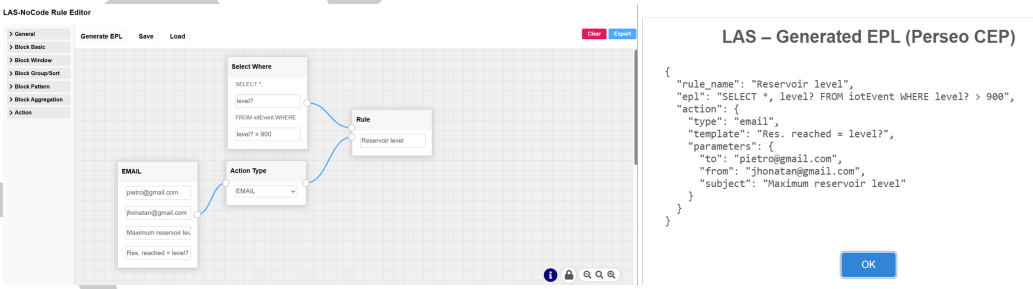

Figure 1: LAS visual rule modeling and automatically generated EPL code compatible with FIWARE Perseo CEP.

Figure 1: LAS-NoCode Rule Editor interface showing visual construction of a CEP rule using connected blocks (left) and the automatically generated EPL rule compatible with Perseo CEP (right), enabling direct deployment to the CEP engine.

Statement of need

CEP engines provide scalable and efficient runtime infrastructures, yet rule authoring remains largely manual and text-based. This creates challenges for:

- Teaching event-driven architectures and CEP concepts.
- Rapid prototyping of event patterns in research scenarios.
- Reproducible specification of rule logic across experiments.

In research contexts, rule definitions are often embedded in configuration files or scripts without structured modeling support, making controlled experimentation and replication more difficult.

LAS addresses this limitation by introducing a visual abstraction layer aligned with CEP semantics. The system:

- Encodes rule logic as structured block-based flows.
- Enforces consistent rule composition patterns.
- Automatically translates visual models into EPL.
- Enables containerized deployment for reproducible environments.
- Integrates natively with Perseo CEP infrastructures within the FIWARE ecosystem.

By formalizing the rule authoring layer, LAS reduces syntactic variability in rule definitions, improves accessibility for students and researchers, and supports reproducible experimentation in event-driven IoT research workflows.

State of the field

CEP has been a recognized research and industrial field since the formalization of event pattern detection models (Luckham, 2002). Engines such as Esper (EsperTech Inc., 2026) provide expressive EPL-based rule definition and efficient in-memory execution. Distributed stream processing frameworks including Apache Flink (Carbone et al., 2015) and Kafka Streams (Kreps et al., 2011) extend event processing to large-scale, fault-tolerant environments.

Within the FIWARE ecosystem, Perseo CEP (FIWARE Foundation, 2026) offers rule-based event processing integrated with context management services. However, these systems primarily focus on runtime execution and scalability rather than structured rule modeling or visual abstraction.

Visual flow-based tools such as Node-RED (Project, 2026) enable event-driven composition through graphical interfaces, but they are not specifically designed for EPL generation or CEP rule modeling.

Previous work by Zimmerle de Lima (Lima, 2017) implemented a Node-RED plugin for Perseo CEP, enabling event-driven mashups for IoT scenarios. The plugin allows users to integrate sensor readings, actuators, dashboards, and alerts through Node-RED's flow-based programming model.

While this approach facilitates rapid prototyping and benefits from Node-RED's active ecosystem, it requires the installation and knowledge of Node-RED and relies on generic flow constructs that do not enforce CEP-specific rule semantics or automated EPL generation.

In contrast, LAS-NoCode Rule Editor provides a standalone domain-specific visual environment for CEP rule construction. Rules are represented as directed block graphs and automatically translated into EPL compatible with Perseo CEP. LAS does not depend on Node-RED or additional platforms, reducing setup complexity and making it more accessible to students and researchers.

This distinction allows LAS to reduce syntactic variability, enforce consistent rule composition patterns, and enhance reproducibility, while preserving the accessibility benefits of visual programming for educational and research contexts.

LAS contributes at the intersection of CEP and low-code tooling by:

1. Providing a domain-specific visual modeling environment tailored to CEP rule construction.
2. Automating the transformation from visual rule graphs into executable EPL code.
3. Supporting reproducible experimental workflows through containerized deployment using Docker (Docker Inc., 2026).

Software design

LAS is implemented as a modular web application composed of:

- Backend: Python with Flask (Pallets Projects, 2026).
- Frontend: JavaScript-based block editor built on Drawflow (jerosoler, 2026).
- Deployment: containerized environment using Docker (Docker Inc., 2026).

Rules are internally represented as directed block graphs. The backend parses the graph into an intermediate representation and generates EPL statements. The generated rules follow the EPL structure expected by Perseo CEP and can be deployed directly to the engine through REST-based integration.

The system can operate as a standalone web application or as a Flask Blueprint embedded into larger systems. Containerization ensures consistent execution across development, experimentation, and teaching environments, supporting reproducibility in research settings.

Acknowledgements

The author thanks the open-source communities behind Flask, Docker, Drawflow, and FIWARE for providing the tools that enabled the development of LAS-NoCode Rule Editor.

References

Carbone, P., Katsifodimos, A., Ewen, S., & Markl, V. (2015). Apache flink: Stream and batch processing in a single engine. *IEEE Data Engineering Bulletin*, *38*(4), 28–38.

Docker Inc. (2026). *Docker: Lightweight linux containers for consistent development and deployment*. https://www.docker.com/

EsperTech Inc. (2026). *Esper complex event processing engine*. https://www.espertech.com/esper/

FIWARE Foundation. (2026). *FIWARE perseo CEP*. https://github.com/telefonicaid/perseo-core

jerosoler. (2026). *Drawflow: Simple flow library*. https://github.com/jerosoler/Drawflow

Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. *NetDB*.

Lima, C. E. Z. de. (2017). *Uso de data flows para processamento de eventos complexos na internet das coisas* [Master's thesis, Universidade Federal de Pernambuco, Centro de Informática]. https://www.cin.ufpe.br/~tg/2017-1/cezl-tg.pdf

Luckham, D. (2002). The power of events: An introduction to complex event processing. *Addison-Wesley*.

Pallets Projects. (2026). *Flask web development framework*. https://flask.palletsprojects.com/

Project, N.-R. (2026). *Node-RED: Flow-based programming for the internet of things*. https://nodered.org.